

T1 Introduksjon og ytelse

T1.1 Datamaskintyper og de 7 store ideene

1.

- Ultralaveffekt systemer
 - På batterier, lite varme
- Innvevde systemer
 - Drone, headset
- Mobiltelefoner
 - Smarttelefoner i egen klasse pga volum av produksjon
- Bærbare datamaskiner
 - Laptops
- Stasjonære datamaskiner
 - Kraftigere men krever mer strøm
- Servere
 - Enda kraftigere og mer lagringsmuligheter eller hoster ting
- Datasentre og superdatamaskiner
 - Svære mengder beregninger og datalagring
 - Hundre tusenvis av prosessorer og brukes for high-end vitenskapelig utvikling og beregninger.

2. De 7 store ideene i datamaskinarkitektur

- Bruk av abstraksjon for å forenkle design
- Gjør det vanlige tilfellet raskt
 - Det som kjører mest sparer vi mest tid på å gjøre mest effektiv
- Ytelse gjennom parallellitet
 - Hvis ytelsen skal opp må vi gjøre flere ting samtidig
- Ytelse gjennom bruk av samlebånd
 - Dele opp oppgavene i steg, gjør alle stegene samtidig men i forskjellige stadier
- Ytelse gjennom prediksjon
 - Gjennom å se hva som har skjedd før, kan vi gjette hva vi skal videre. Har mekanisme for å rydde opp om vi gjetter feil.
 - Funker bra om en er god å gjette
- Lag et hiarki av minneenheter
 - Skape illusjon av et raskt og stort minne

- Pålitelighet gjennom redudans
 - Er systemet stort nok, så vil statistisk sett noe ryke hele tiden. Redudans sier da at om en ting skulle ryke så må en annen ting kunne ta over.

3

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%
ronnabyte	RB	10^{27}	robibyte	RiB	2^{90}	24%
queccabyte	QB	10^{30}	quebibyte	QiB	2^{100}	27%

Originalt var ting oppgitt i binære term men på grunn av standard desimalsystemet har det blitt vanlig å bruke desimale term istedet.

T1.2 Under overflaten

1.

- Applikasjonsprogramvare
 - Programvaren som bygger opp diverse applikasjoner. Kjøres på systemprogramvaren til datamaskinen.
 - Kan være alt fra kompliserte CAD programmer til enkle teksteditorer.
- Systemprogramvare
 - To hoved systemprogramvarer som alle datamaskiner i dag bruker:
 - Operativsystemet
 - IO mellom brukers program og maskinvaren.
 - Har flere sikkerhet og tilsyns funksjoner som, utdeling av minne og lagring, passe på at flere programmer kjører samtidig og IO.
 - Eks: Windows, Linux, Android og iOS.
 - Kompilatoren
 - Oversetter mellom høy nivå kodespråk som C, C++ og Python til maskinintstruksjoner som maskinvaren kan kjøre.
 - Kan være

2.5 Hovedkomponenter i en datamaskin

- En datamaskin består av fem komponenter:
 - Inndata
 - Utdata
 - Minne
 - Kontrollenhet (setter opp datastiene)
 - Datasti
- De to siste blir ofte slått sammen til Prosessor som i 3 ↓

3. Prinsippet om lagrede program (stored program concept)

Minne: inneholder instruksjoner og data

- Ser ikke forskjell på de, bare nullere og enere

Prosesor (CPU): Gjør beregninger på data gitt instruksjoner den mottar

- Henter en instruks fra minnet og eventuell annen data den trenger.
- Gir data tilbake til minne når ferdig.

Inndata/Utdata (I/O):

- Kommuniserer med omverdenen (skjerm, tastatur, nettverk, etc.)

4. Produksjonsprosessen for IC-er

- Silisiumspølser blir kuttet opp i tynne silisiumskiver (wafer).
- Skivene blir sendt igjennom en serie med prosesserings steg hvor kjemikalier blir brukt til å lage mønster til transistorer, kondensatorer og isolatorer.
- De forskjellige delene av skivene blir så testet, kuttet og separert de fungerende delene. I produksjon av noen IC-er som CPU-er blir de gradert og skilt utifra hvor mye av skiven som fungerer, som gir f.eks. en I5 vs en I7 Intel prosessor.
- Delene blir så pakket inn i pakning før de testes igjen og sendes ut til kunder.

T1.3 Ytelse

1. Ytelsesmetrikkene i datamaskinarkitektur

- Kjøretid (Execution Time): Tiden det tar å fullføre en oppgave.
- Gjennomstrømning (Throughput): Mengden arbeid fullført per tidsenhet.
- Effektforbruk: Raten datamaskinen forbruker energi, som øker med klokkefrekvens og spenning.
- Ytelse: Invers av kjøretid; høyere ytelse betyr lavere kjøretid

2. Kjøretid og båndbredde

- Kjøretid
 - Tid mellom start og slutt av en oppgave
 - Også kalt execution time.
 - Tid for enkelte oppgaver, vil at den vanlige oppgaven skal være rask.
- Gjennomstrømming
 - Også kalt båndbredde.
 - Hvor mye arbeid som kan bli gjort på en bestemt tid.
 - Parallelle systemer vil oppnå større gjennomstrømming.

3. Iron law

3. Forklaring av «The Iron Law»

The Iron Law of Processor Performance

Denne loven uttrykker kjøretid som en funksjon av tre faktorer:

$$\text{Kjøretid} = \frac{\text{Instruksjoner}}{\text{Program}} \times \frac{\text{Klokkeysikler}}{\text{Instruks}} \times \frac{\text{Sekunder}}{\text{Klokkesykler}} = \frac{\text{Sekund}}{\text{Program}}$$

$$CPI = \frac{\text{Clock cycles}}{\text{Instruction}}$$

Hva betyr dette?

4. Instructions per Program

1. Antall instruksjoner i et program. Reduseres ved å optimalisere kode eller bruke kompilatorer som fjerner ineffektivitet.

5. Cycles per Instruction (CPI)

1. Avhenger av maskinens arkitektur. Reduseres ved å bruke raskere instruksjonssett eller parallell utførelse.

6. Time per Cycle

- Avhenger av klokkefrekvensen. Kortere syklustid gir høyere klokkefrekvens.

Hvordan bruke Iron Law?

- Eksempel 1: Hvis klokkefrekvensen økes, reduseres tiden per syklus. Dette gir raskere kjøretid, men kan øke strømforbruk.
- Eksempel 2: Redusere antall instruksjoner per program gjennom bedre algoritmer

Hardware or software component	Affects what?	How?
Algorithm	Instruction count, CPI	The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more divides, it will tend to have a higher CPI.
Programming language	Instruction count, CPI	The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher CPI instructions.
Compiler	Instruction count, CPI	The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in varied ways.
Instruction set architecture	Instruction count, clock rate, CPI	The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.

4. Spenning og klokkefrekvens => effektforbruk og strømbruk

$$Energy \propto \frac{1}{2} \cdot \text{Capacitive load} \cdot Voltage^2$$

$$Power \propto \frac{1}{2} \cdot \text{Capacitive load} \cdot Voltage^2 \cdot \text{Frequency switched}$$

Problemet med energi er jo høyere frekvens de har på klokken jo mer varme vil genereres.

5. Hva er en testprogramsamling?

En samling av programmer og oppgaver som brukes til å evaluere ytelsen til en datamaskin.

- Hvorfor brukes de?
 - Gir en standardisert måte å måle ytelse på.
 - Brukes til å sammenligne ulike maskiner, arkitekturer eller konfigurasjoner.
 - Identifiserer flaskehalser i systemet.

Eksempler på testprogramsamlinger

1. SPEC (Standard Performance Evaluation Corporation)
 1. Tester CPU, minne og I/O-ytelse.
 2. Brukes ofte i akademia og industri.
2. TPC (Transaction Processing Performance Council)

1. Tester database- og transaksjonsytelse.
3. LINPACK
 1. Brukes til å måle ytelsen til superdatamaskiner ved å evaluere flyttallsoperasjoner

T1.4 Parallelle datamaskiner

1.

Stort sett alle datamaskiner i dag har mer enn en prosessorkjerne fordi det fører til høyere ytelse. Dette kommer av at det er mer energieffektivt og øker ytelse ved flere applikasjoner og tråder kan kjøres parallellt over flere prosessorer.

2.

Amdahls lov

$$\text{Execution time after improvement} = \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected}$$

- Gevinsten av enhver optimalisering er alltid begrenset av det du ikke optimaliserer
 - Derfor må vi måle først og optimalisere etterpå
 - ... altså bruke den kvantitative metoden
 - Vi får mest igjen for innsatsen vår når vi fokuserer på det vanlige tilfellet («make the common case fast»)

T2 Introduksjonssett

T2.1 Instruksjoner

1

- Designprinsipp 1: Regelmessighet forenkler
 - Alle instruksjoner spesifiserer alltid en operasjon og tre operander.
 - eks: add, rd, rs1, rs2

- Designprinsipp 2: Mindre er raskere
 - Hvis vi har flere registre så kan vi lagre mer, men da tar det lengre tid å akksessere de som gjør at vi må øke sykeltiden. Da er det mer effektivt å ha færre registre, typisk 32 stykk.
- Designprinsipp 3: Gode løsninger krever gode kompromiss
 - Ved å ha faste mønster på f.eks instruksjoner i IAS så forenkler det prosessen å bestemme hva ulike instrukser er. F.eks er Opcode alltid 7 bits og på samme sted i instruksjonen som gjør det lett å plukke den ut.
 - Vi har flere instruksjoner som gjør forskjellige ting bra, men prøver å holde de så like som mulige (R-, I- og S-type)

2.

- Oversette fra høynivåspråk til assemblyinstruksjoner (og omvendt):

3.

- Oversette assemblyinstruksjoner til maskinkode (og omvendt):
 - Sjekk riktig Opcode og funct3 for instruksjonstypen
 - Sjekk enten for funct7 for R-types eller konstanter/addresser for I- og S-type
 - Se hva registre den henter fra og skriver til
 - Done

4.

- R-type: Brukes for operasjoner mellom registre (f.eks.:add,:sub).
 - funct7 $\Rightarrow$ 7 bits
 - rs2 $\Rightarrow$ 5 bits
 - rs1 $\Rightarrow$ 5 bits
 - funct3 $\Rightarrow$ 3 bits
 - rd $\Rightarrow$ 5 bits
 - opcode $\Rightarrow$ 7 bits
- I-type: Brukes for instruksjoner med umiddelbare konstanter
 - immediate $\Rightarrow$ 12 bits
 - rs1 $\Rightarrow$ 5 bits
 - funct3 $\Rightarrow$ 3 bits
 - rd $\Rightarrow$ 5 bits
 - opcode $\Rightarrow$ 7 bits
- S-type: Brukes for lagring til minne (sw).
 - immediate[11:5] $\Rightarrow$ 7 bits
 - rs2 $\Rightarrow$ 5 bits

- rs1 $\Rightarrow$ bits
- funct3 $\Rightarrow$ 3 bits
- immediate[4:0] $\Rightarrow$ 5 bits
- opcode $\Rightarrow$ 7 bits
- U-type: Brukes for store konstanter og minneadministrasjon (lui)

5. Instruksjoner lagres i minne og kontrollflyt

- Instruksjoner lagres som 32 bit ord i minne som adresseres til ved bruk av point counter (PC).
- Vi kan bruke betingede og ubetingede hopp (beq, neq vs jal, jalr) for å hoppe mellom instruksjoner.
 - Viktig å passe på å lagre og hente ut fra minnet (stacken) når bruker funksjoner/betingede hopp som overskriver verdier i registrene en bruker!

T2.2 Heltall og logiske operasjoner

1. Gjøre om mellom 2, 10 og 16 tall systemet

$$64_{10} = 1000000_2 = 40_{16} \quad 26_{10} = 11010_2 = 1A_{16}$$

2. 2 komplement til desimal

$$101100_2 = -20_{10}$$

3. Mer om 2 komplement

- Det beholder samme tallverdi fordi om det ikke er ment å være negativt så slenger du på en 0 foran, og om det ikke er plass til det så har du overflyt.

4. Overflyt

- Overflyt er når en prøver å legge til et tall slik at det blir høyere enn hva det kan lagres som.
 - Da vil den overflyte og gå til å bli enten veldig liten eller veldig negativ om bruker 2 komplement.

5. Logiske operasjoner på binære tall

$$1 \cdot 1 = 1 \quad 1 + 0 = 1 \quad 1 \cdot 0 = 0$$

- Logisk og aritmetisk bitskifting

Logical operations	C operators	Java operators	RISC-V instructions
Shift left	<<	<<	sll, slli
Shift right	>>	>>>	srl, srli
Shift right arithmetic	>>	>>	sra, srai
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit XOR	^	^	xor, xori
Bit-by-bit NOT	~	~	xori

Av og til er det nyttig å fikle med individuelle bit

Eksempler: AND kan brukes til å maskere biter og shifting mot venstre kan brukes til multiplikasjon med 2

T2.3 Funksjonskall

1. Funksjonskall og kallkonvensjon

For å kjøre en funksjon, må vi gjennom følgende oppgaver:

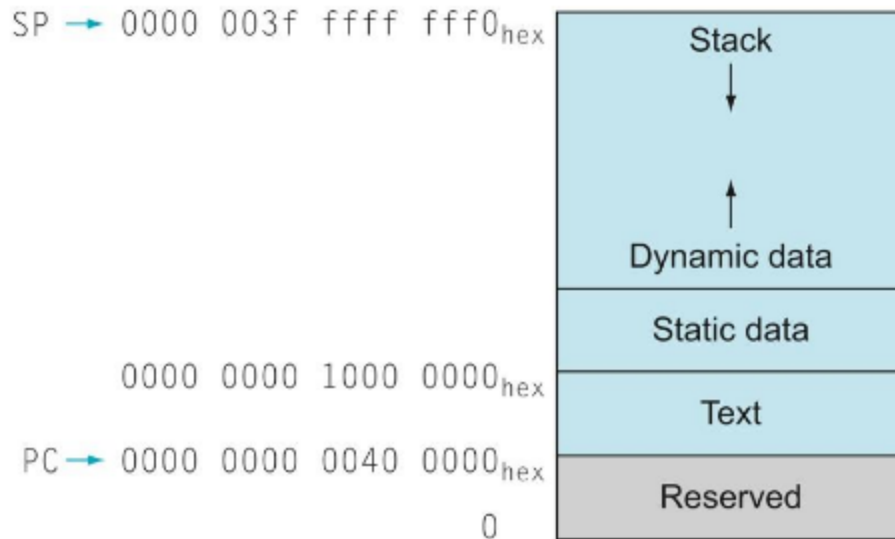
1. Legg parameterne et sted funksjonen kan få tak i dem
2. Overfør kontroll til funksjonen
3. Allokere nødvendige ressurser (registre, minne)
4. Utfør oppgaven
5. Legg returverdien et sted der den som kalte funksjonen kan få tak i dem
6. Overfør kontroll til kallstedet

Vi bruker funksjonskall for å kunne gjenbruke kode som skal kjøre flere ganger, som løkker eller spesifikke beregninger.

Kallkonvensjon

- For å sikre interoperabilitet mellom kompilatorer, bibliotek, etc. må vi ha regler for hvilken tilstand som bevares etter funksjonskall
- Temp registre og argument registre vil ikke bevares, mens alt annet må bevares når en kaller på funksjoner. Gjør dette ved å lagre til minne.

2. Minnekart i RISC-V



Figur 2.13

- Minnekart beskriver hvordan adresserommet skal brukes:
 - Programmet lastes inn i «Text» segmentet
 - Statiske data (for eksempel globale variabler) legges i «Static data»
 - Dynamiske data (malloc og new) kalles «heap» og vokser mot høyere adresser
 - Stakken («Stack») brukes til funksjonskall og vokser mot lavere adresser
 - Hvis stakken møter dynamisk allokerende data, får vi «stack overflow»

3. Statisk vs dynamisk data

- Statisk data:
 - Bestemmes når programmet kompileres
 - Bruker stakken
- Dynamisk data:
 - Bestemmes når programmet kjøres

T2.4 Instruksjoner, diverse

1. Tekst

Bokstaver og tekst representeres ofte med ASCII

- Bruker 8 bits (1 byte) per karakter
- Lagres som U-type
 - Immediate[31:12] rd opcode
 - 20 bit 5 bits 7 bits

2. Unngå hoppe langt

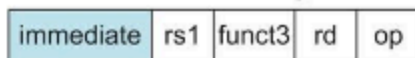
Ved å bruke PC relativ hopping kan vi enkelt bevege oss gjennom instruksjoner.

- PC lagrer instruksjonsadressen til nåværende instruksjon, som vi kan $\pm 4x$ for å bevege oss rundt.

3. RISC-V 4 Adressemodi

- Måten vi kan oppgi en adresse på kalles en adressemodus
- RISC-V støtter fire ulike adressemodi:
 - Konstantbasert adressering
 - Registerbasert adressering
 - Grunnadressebasert adressering
 - PC-relativ adressering

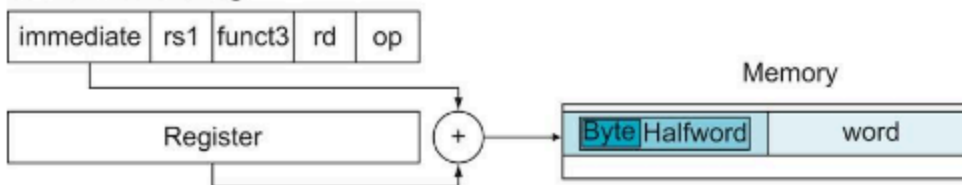
1. Immediate addressing



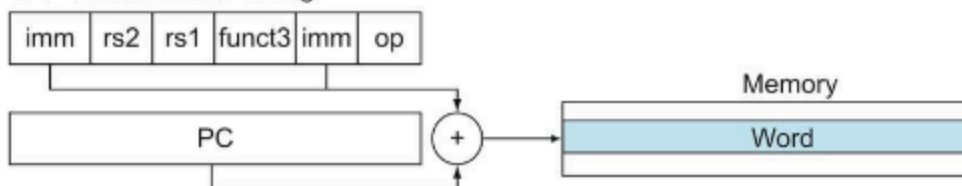
2. Register addressing



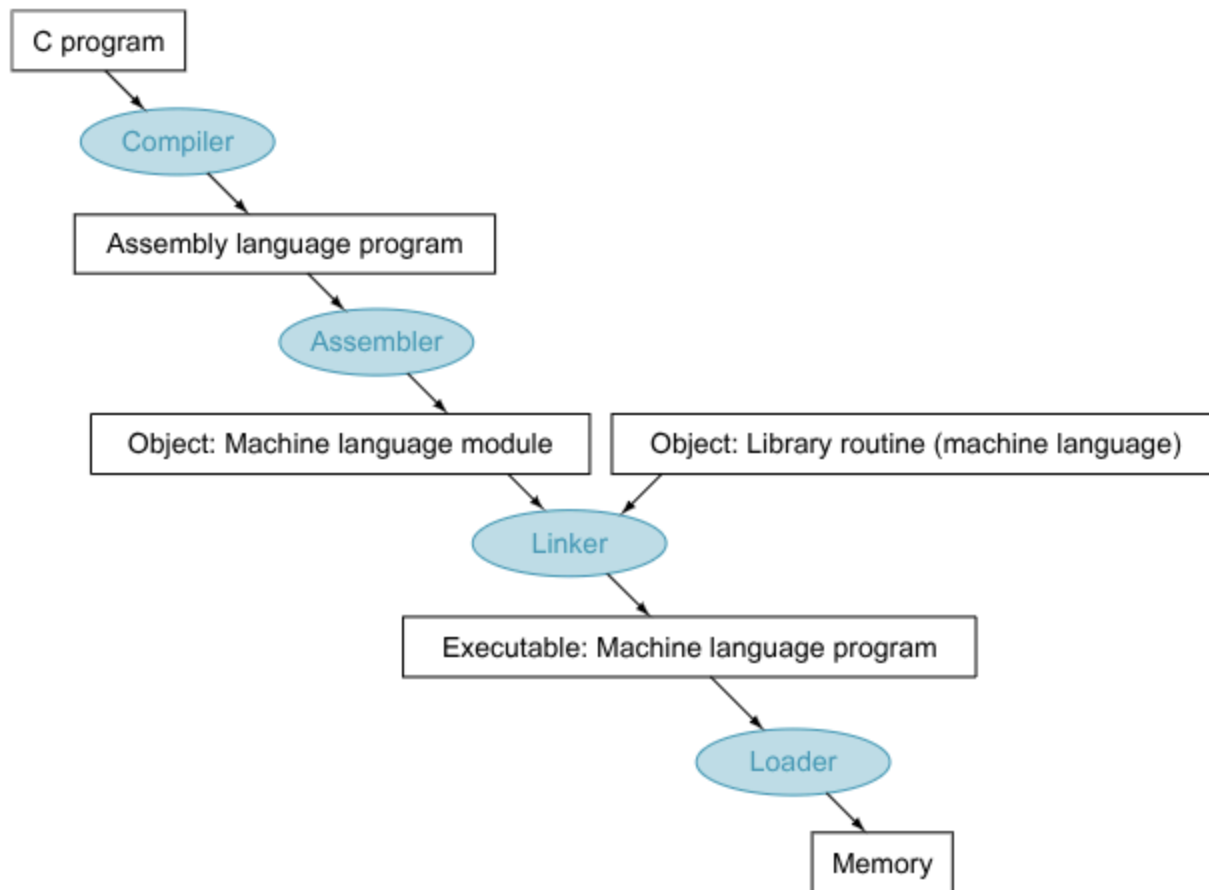
3. Base addressing



4. PC-relative addressing



4. Forklar denne



- C programmet blir gjort om til assemblykode av kompilatoren
- Assembleren tar assemblykoden og gjør det om til maskinspråk
- Linkeren slår sammen maskinspråket og de delene av eventuelle bibliotek brukt som også er i maskinspråk til et kjørbart program i maskinspråk.
- Programmet blir sendt til en laster modul som laster programmet til minnet.

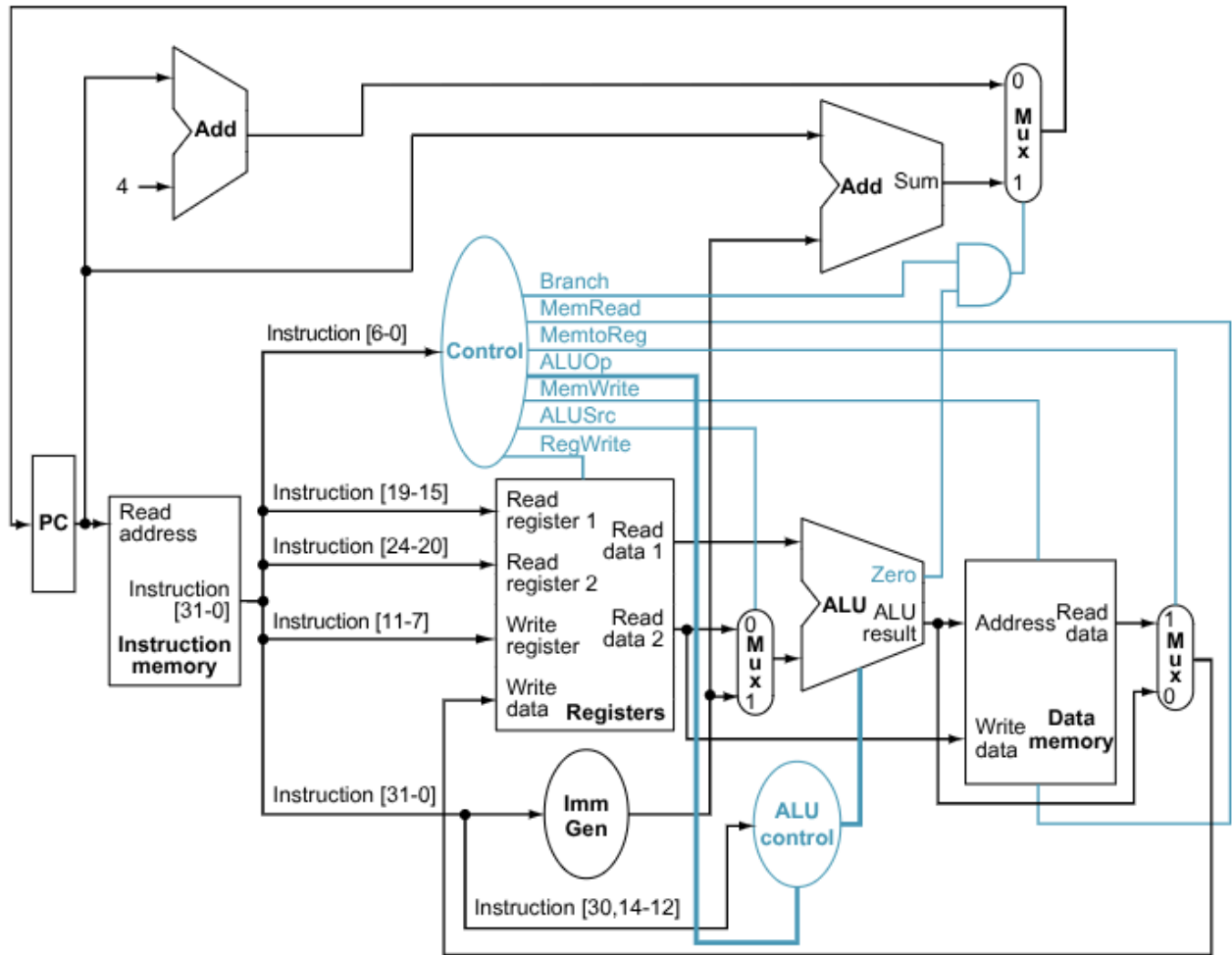
T3: Enkelsykelprosessor

T3.1 Enkeltskykel-prosessor

1. Datasti og kontrollenhet

- Datasti
 - Navnet på veien dataen tar for å gjennomføre instruksjonen
- Kontrollenhet
 - Basert på operasjonskodene i instruksjonen omgjør kontrollenheten datastien slik at den samsvarer med instruksjonen som skal gjennomføres.

2. Forklar mikroarkitektur til enkeltsykel-prosessor bilde



3. Kontrollord ("don't care")

- Kontrollord er signalene som går ut av kontrollenheten basert på operasjonskodene i instruksjonen.
 - Eks:
 - add: 000-10-001
 - lw: 011-00-011
 - sw: 00X-00-110
 - beq: 10X-01-000

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

4. Identifisere instruksjonstype fra kontrollord

Se over^^

T3.2 Kombinatorisk logikk

1. AND, OR, NOT

Ade pensum

2. NAND, NOR, MUX osv

Ade pensum

3. Boolske uttrykk og sannhetstabeller

Ade pensum

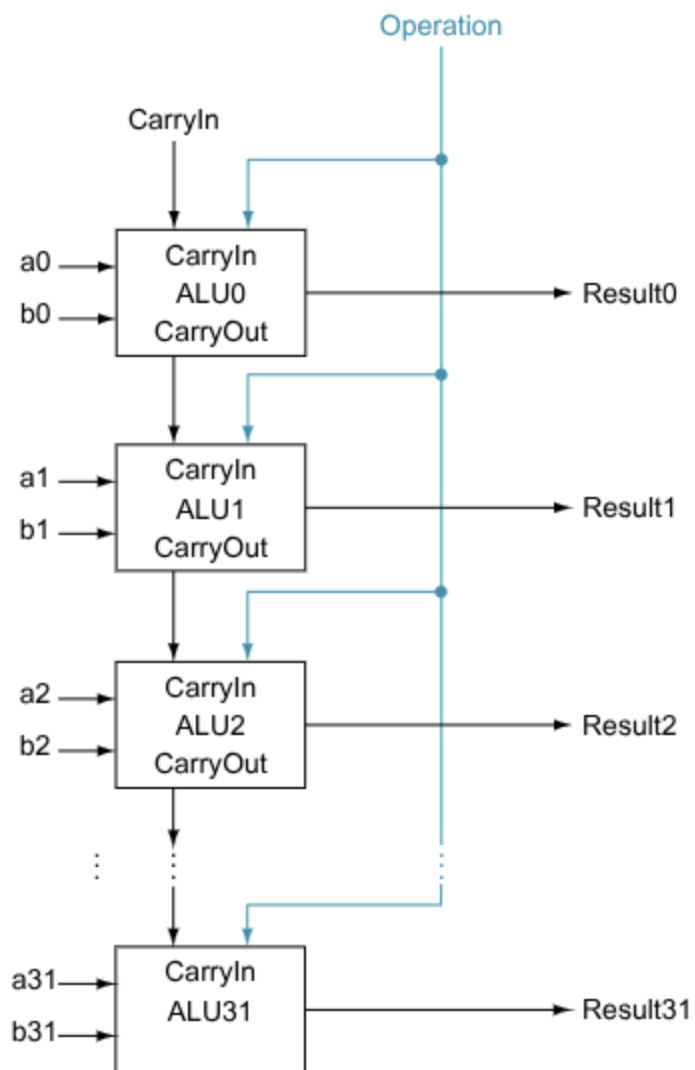
4. Buss & notasjonen for adressering av enkeltlinjer

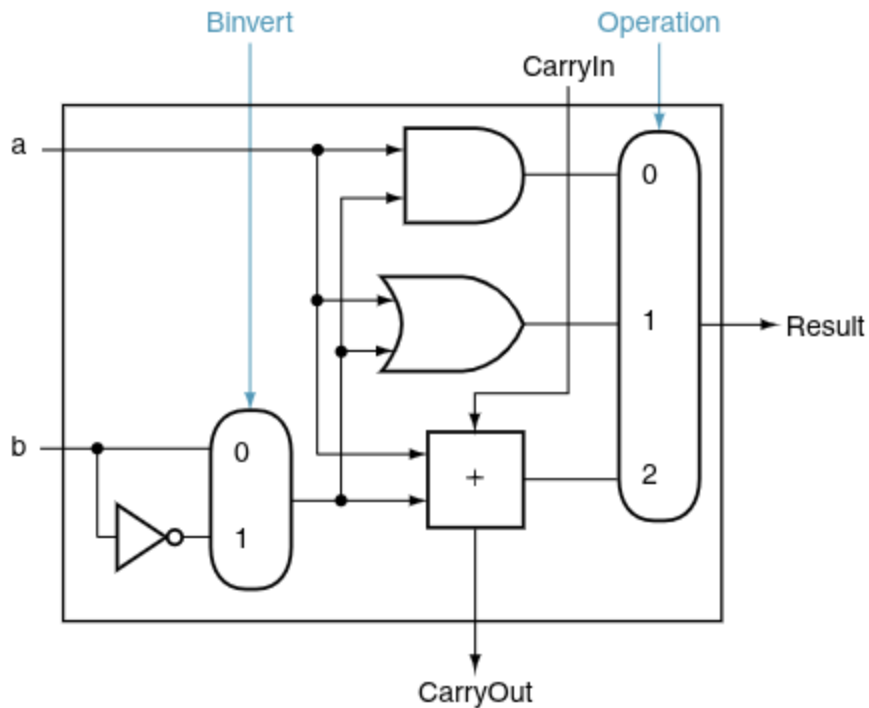
Busser er samlinger av datalinjer som blir behandlet som et enkelt logisk signal eller bare en samling av datalinjer med forskjellige bruksområder og kilder.

- Om en bus ikke er 32 bit er det eksplesitt merket.
- Adresserer enkeltlinjer ved å si hvilket nummer det er av totale linjene.
 - Eks: MUX med 32 bit bus vil egt være 32 2:1 MUXer med IOer A0, B0, C0 → A31, B31, C31

T3.3 Aritmetisk-logisk enhet (ALU)

1.





- Vi stacker 32 1 bit ALUer for å få en 32 bit ALU. Da kobles operationslinjen til alle ALUene og CarryIn fra minst signifikante bit kobles stigende til mest signifikante bit.
- Innen i en 1 bit ALU er det en 3:1 MUX som styres av operation. Den velger mellom at de to inngangssignalene skal i en AND, OR eller i en adderer. Om vi inverterer b med Binvert=1 og har CarryIn=1 så vil addereren utføre toers komplement med b-a.
- Om subtraksjon gir 0 så vil Zero gå høy og som brukes til å detektere f.eks. om en beq skal branche eller ikke.

2.

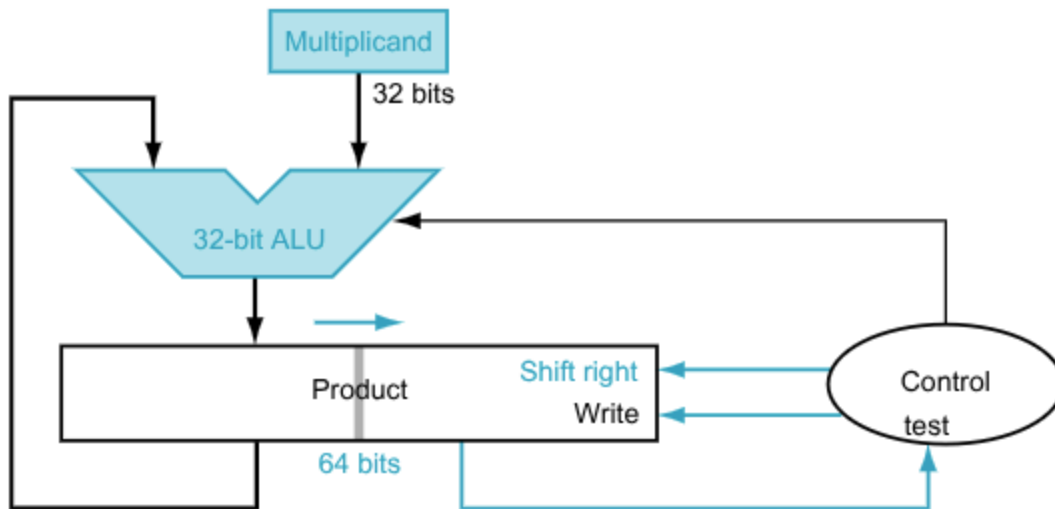


FIGURE 3.5 Refined version of the multiplication hardware. Compare with the first version in Figure 3.3. The Multiplicand register and ALU have been reduced to 32 bits. Now the product is shifted right. The separate Multiplier register also disappeared. The multiplier is placed instead in the right half of the Product register, which has grown by one bit to 65 bits to hold the carry-out of the adder. These changes are highlighted in color.

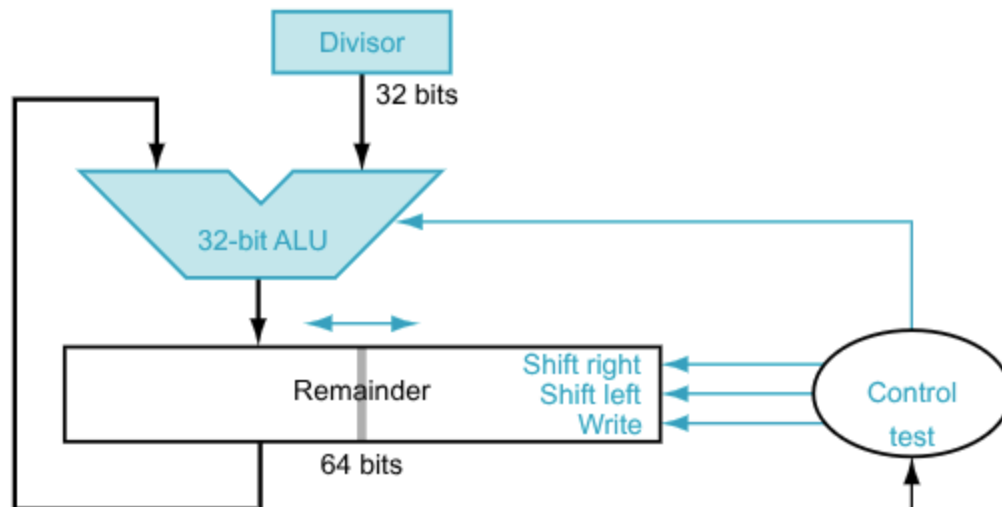


FIGURE 3.11 An improved version of the division hardware. The Divisor register, ALU, and Quotient register are all 32 bits wide. Compared to Figure 3.8, the ALU and Divisor registers are halved and the remainder is shifted left. This version also combines the Quotient register with the right half of the Remainder register. As in Figure 3.5, the Remainder register has grown to 65 bits to make sure the carry out of the adder is not lost.

3. Fixed point

- Vi definerer en fast plass hvor vi plasserer komma for å presentere flyttall.
- Fordel:

- Vi kan behandle flyttall likt som heltall.
- Ulempe
 - Vi må velge mellom å ha store tall eller presise tall.

4. Hvorfor flyttall og addisjon og multiplikasjon i maskinvare

Vi vil ha måte å representere reelle tall som pi, e og generelt desimaler. Vi oppnår dette med flyttall som består av 3 deler. Fortegnsbit (S), Brøkdel (F) og Eksponenten (E) som gir den generelle formelen:

$$(-1)^S \times F \times 2^E$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

IEEE 754: Bias=127

$$(-1)^S \times (1 + \text{FRACTION}) \times 2^{\text{Exponent}-\text{Bias}}$$

ADDISJON

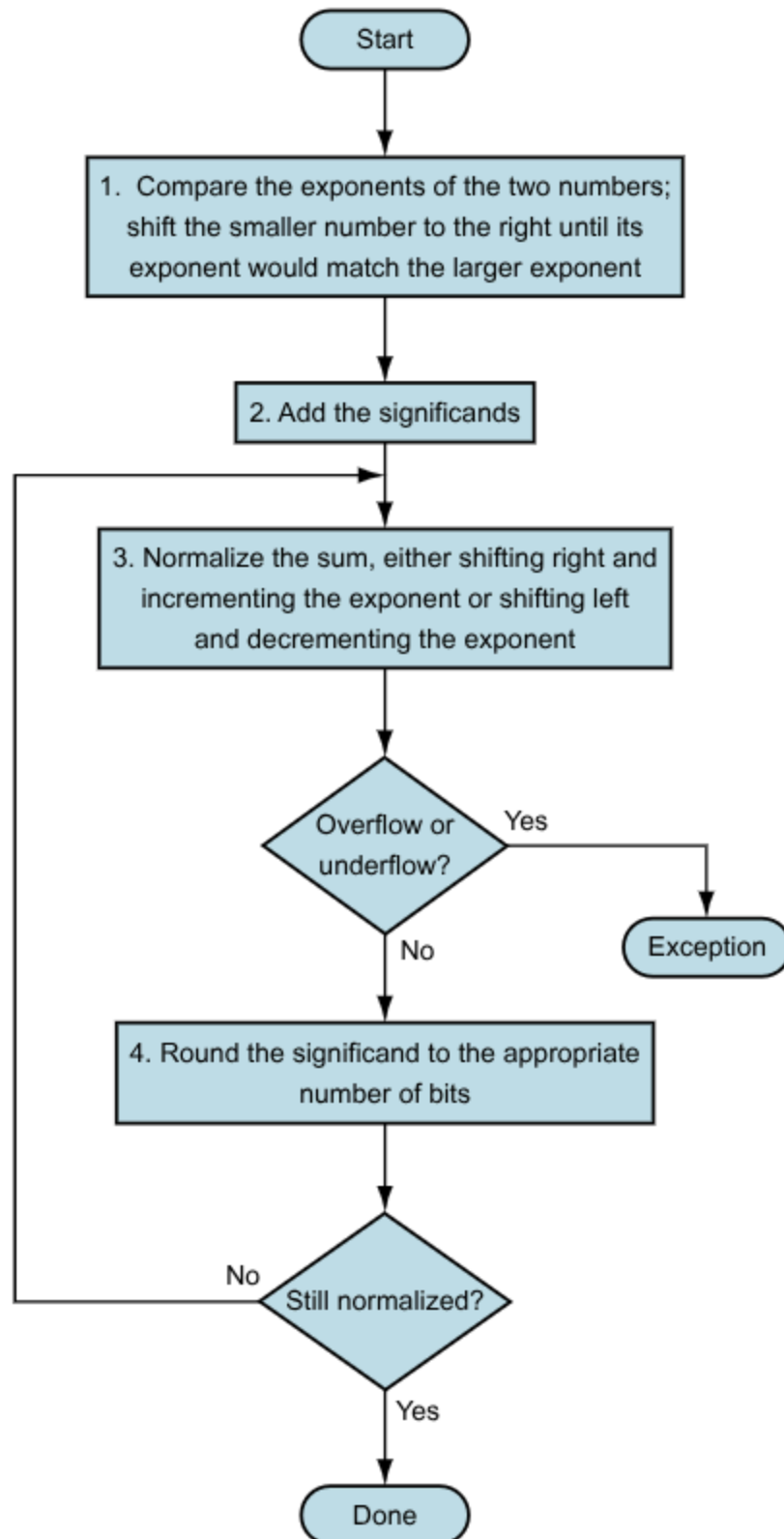


FIGURE 3.14 Floating-point addition. The normal path is to execute steps 3 and 4 once, but if rounding causes the sum to be unnormalized, we must repeat step 3.

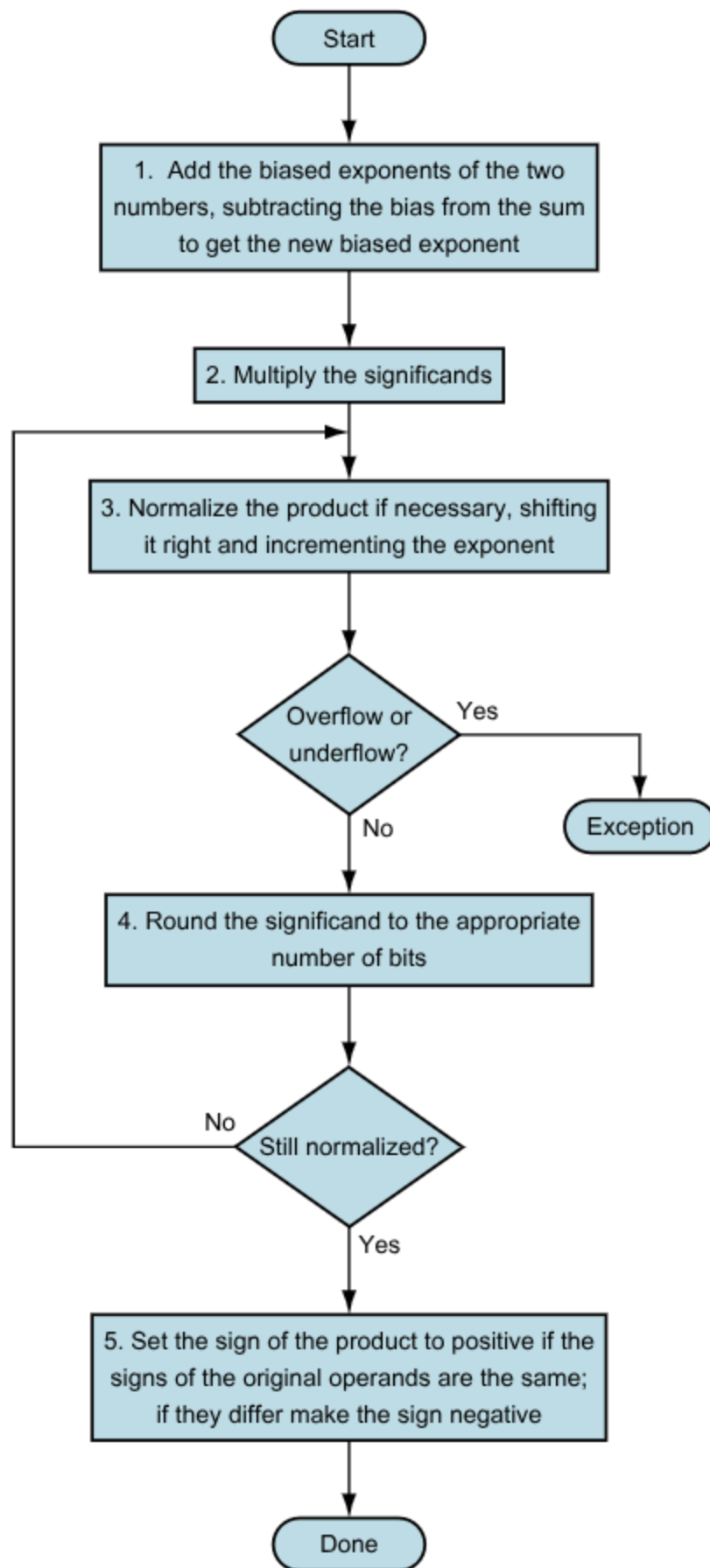


FIGURE 3.16 Floating-point multiplication. The normal path is to execute steps 3 and 4 once, but if rounding causes the sum to be unnormalized, we must repeat step 3.

5. Desimaltal til flyttal

Converting Binary to Decimal Floating Point

EXAMPLE

What decimal number does this single precision float represent?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The sign bit is 1, the exponent field contains 129, and the fraction field contains $1 \times 2^{-2} = 1/4$, or 0.25. Using the basic equation,

ANSWER

$$\begin{aligned} (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} &= (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)} \\ &= -1 \times 1.25 \times 2^2 \\ &= -1.25 \times 4 \\ &= -5.0 \end{aligned}$$

6. SIMD-instruksjoner

- Ved bruk av SIMD (Single Input Multiple Data streams) kan vi legge til ekstra komponenter slik at en 32 bits ALU kan deles og brukes som en mindre ALU dersom det kun trengs f.eks. 2-bits ALU.
- Om hver av de enkelte ALU-ene har en form for aktiveringssekvens for inputen så kan vi styre i inputen hvor mange ALU-er vi vil bruke om gangen og til hva, slik at vi kan kjøre flere operasjoner i parallell.
- Det er en liten ekstra kostnad som gir stor økt ytelse.

T4 Flersykkelpro세서

T4.1 Flersykkelprosessor

1. Forklare mikroarkitektur til flersykkelprosessor

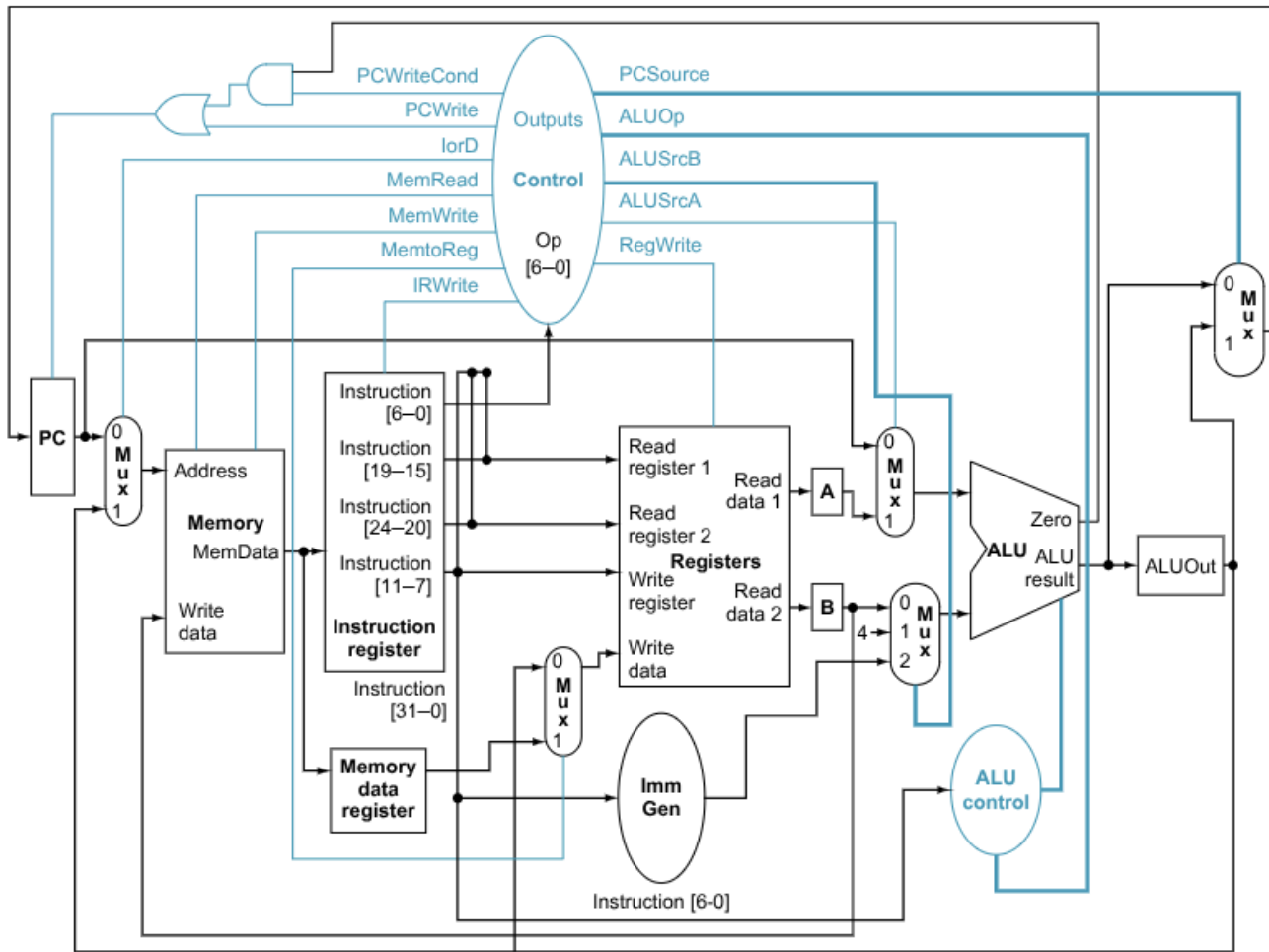


FIGURE e4.5.4 The complete datapath for the multicycle implementation together with the necessary control lines. The control lines of Figure e4.5.3 are attached to the control unit, and the control and datapath elements needed to effect changes to the PC are included. The major additions from Figure 4.29 include the multiplexer used to select the source of a new PC value; gates used to combine the PC write signals; and the control signals PCSource, PCWrite, and PCWriteCond. The PCWriteCond signal is used to decide whether a conditional branch should be taken.

Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None.	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None.	Memory contents at the location specified by the Address input is replaced by the value on the Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
lorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None.	The output of the memory is written into the IR.
PCWrite	None.	The PC is written; the source is controlled by PCSrc.
PCWriteCond	None.	The PC is written if the Zero output from the ALU is also active.

Actions of the 2-bit control signals

Signal name	Value (binary)	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the immediate generated from the IR.
PCSrc	00	Output of the ALU ($PC + 4$) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address (IR[25:0] shifted left 2 bits and concatenated with $PC + 4[31:28]$) is sent to the PC for writing.

FIGURE e4.5.5 The action caused by the setting of each control signal in Figure e4.5.4 on page 323. The top table describes the 1-bit control signals, while the bottom table describes the 2-bit signals. Only those control lines that affect multiplexors have an action when they are deasserted. This information is similar to that in Figure 5.16 on page 306 for the single-cycle datapath, but adds several new control lines (IRWrite, PCWrite, PCWriteCond, ALUSrcB, and PCSrc) and removes control lines that are no longer used or have been replaced (PCSrc and Branch).

Vi deler opp instruksjons-utføringen i flere skritt:

1. Instruksjonshenting [IF]
2. Instruksjonsdekoding og [ID] registerlesing
3. Utføring, adresseberegning, eller fullføring av forgreiningsinstruksjoner [EX]
4. Minneaksess eller fullføring av R-type instruksjoner [MEM]
5. Fullføring av minnelesing [WB]

2. Kontrollenhet som tilstandsmaskin

- Kontrollenheten vil bli en tilstandsmaskin fordi den vil oppdatere og endre seg for klokkesyklene basert på hva tilstand forskjellige instruksene er i. Den må kunne gjøre klar f.eks for å lese av registrene til en add instruks samt skrive til minne en sw.

3. Kontrollord beskriver tilstanden i tilstandsmaskinen

- Fra kontrollordet kan vi se hva tilstand tilstandsmaskinen er i utifra hvor langt i kontrollordet forskjellige instruksjonstyper har kommet.

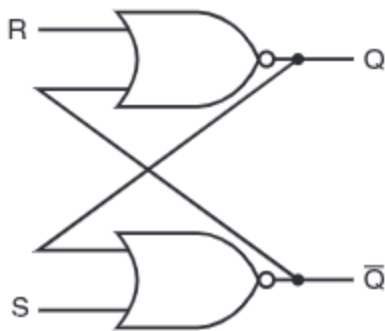
- F.eks. vil en add instruks i 3 klokkesykel være ulik en minnebasert eller branch instruks.

4. Fordeler og ulemper med enkelt- og flersykelprosessor

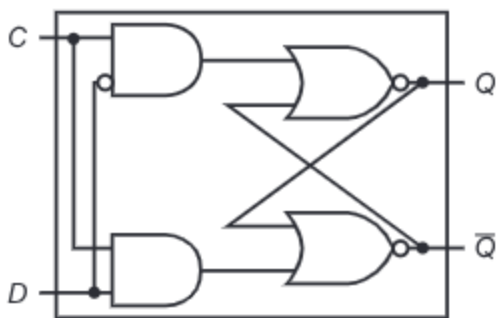
- Enkeltsykel
 - Lang kritisk sti.
 - Må ha flere adderere og minner for å få alt i en klokkesykel.
 - Tar mer areal uten bedre ytelse.
 - Vanskelig med operasjoner som krever flere klokkesykler som divisjon.
 - Alle instruksjoner tar like lang tid som er ikke god bruk av "make the common case fast".
- Flersykel
 - Kan gjøres svært liten og billig.
 - Bruker flere klokkesykler på en instruksjon, CPI øker.
 - Mindre å gjøre per klokkesykel gjør at vi kan øke klokkefrekvens.
 - Ikke særlig god ytelse.

T4.2 Sekvensiell logikk

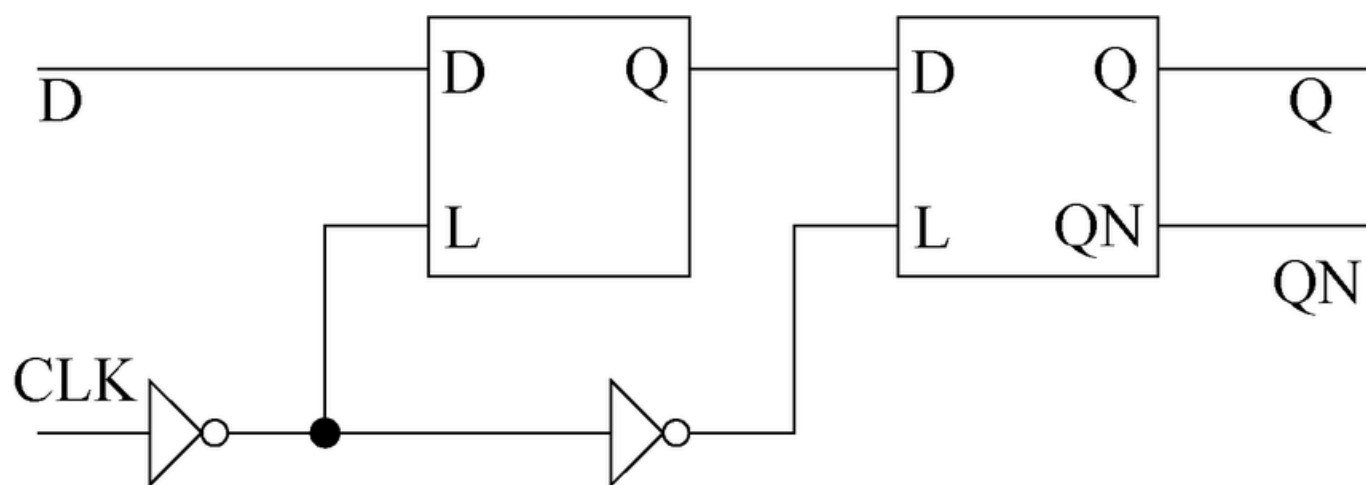
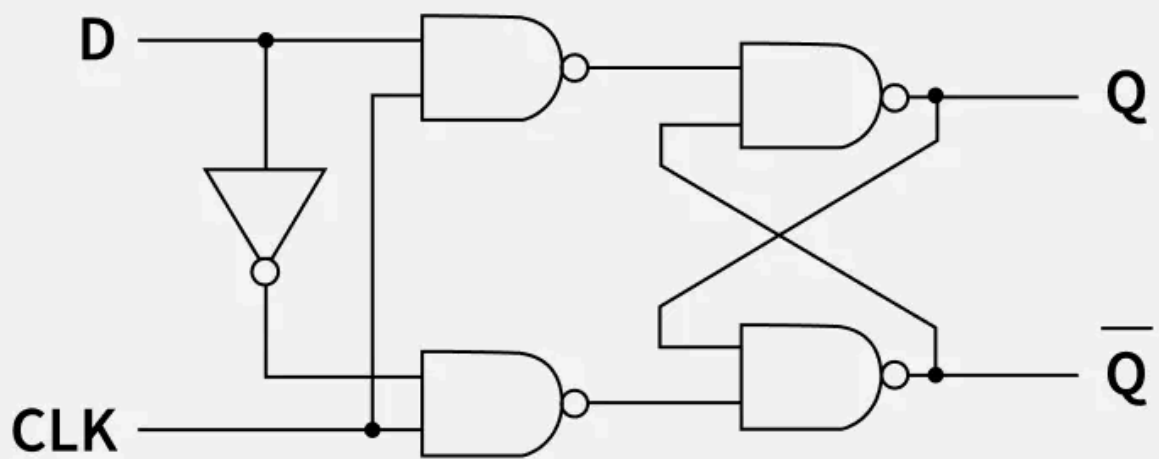
1. SR-lås, D-lås og D-vippe



D-lås (D latch)



D-vippe



2. Konstruere registre og registerfiler

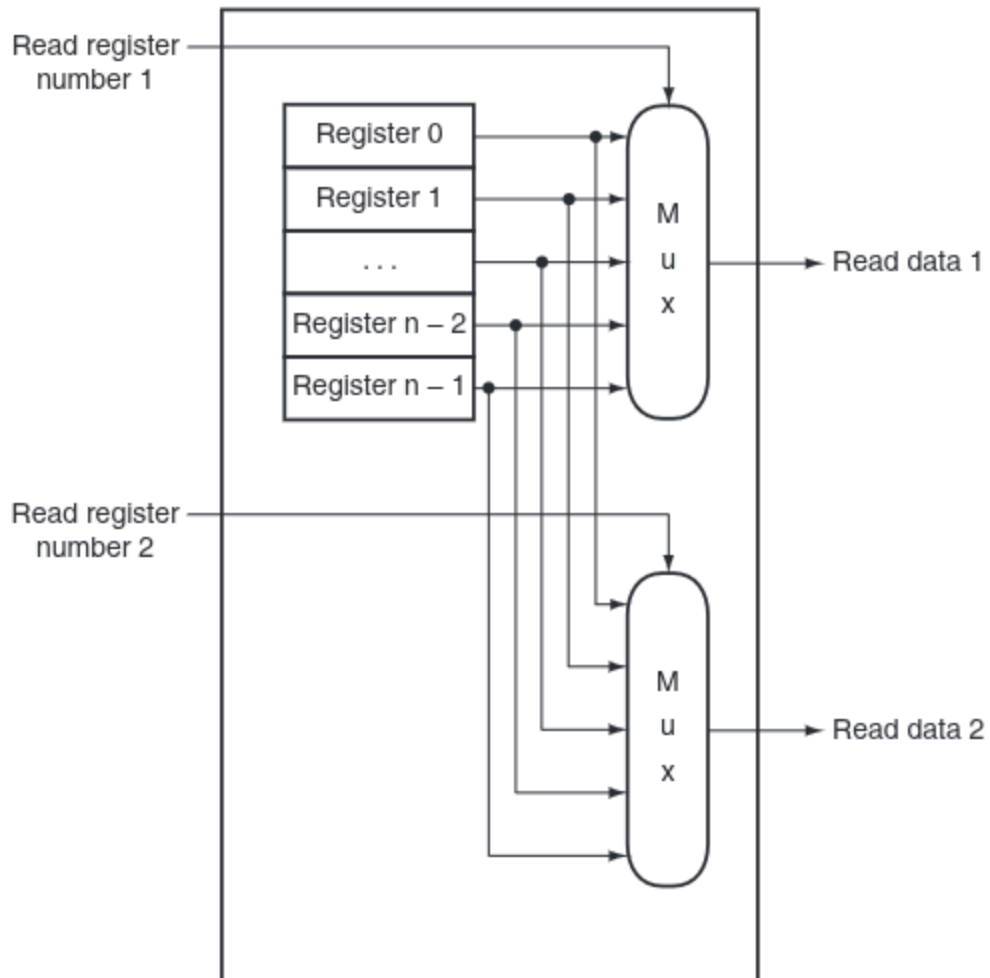


FIGURE A.8.8 The implementation of two read ports for a register file with n registers can be done with a pair of n -to-1 multiplexors, each 32 bits wide. The register read number signal is used as the multiplexor selector signal. [Figure A.8.9](#) shows how the write port is implemented.

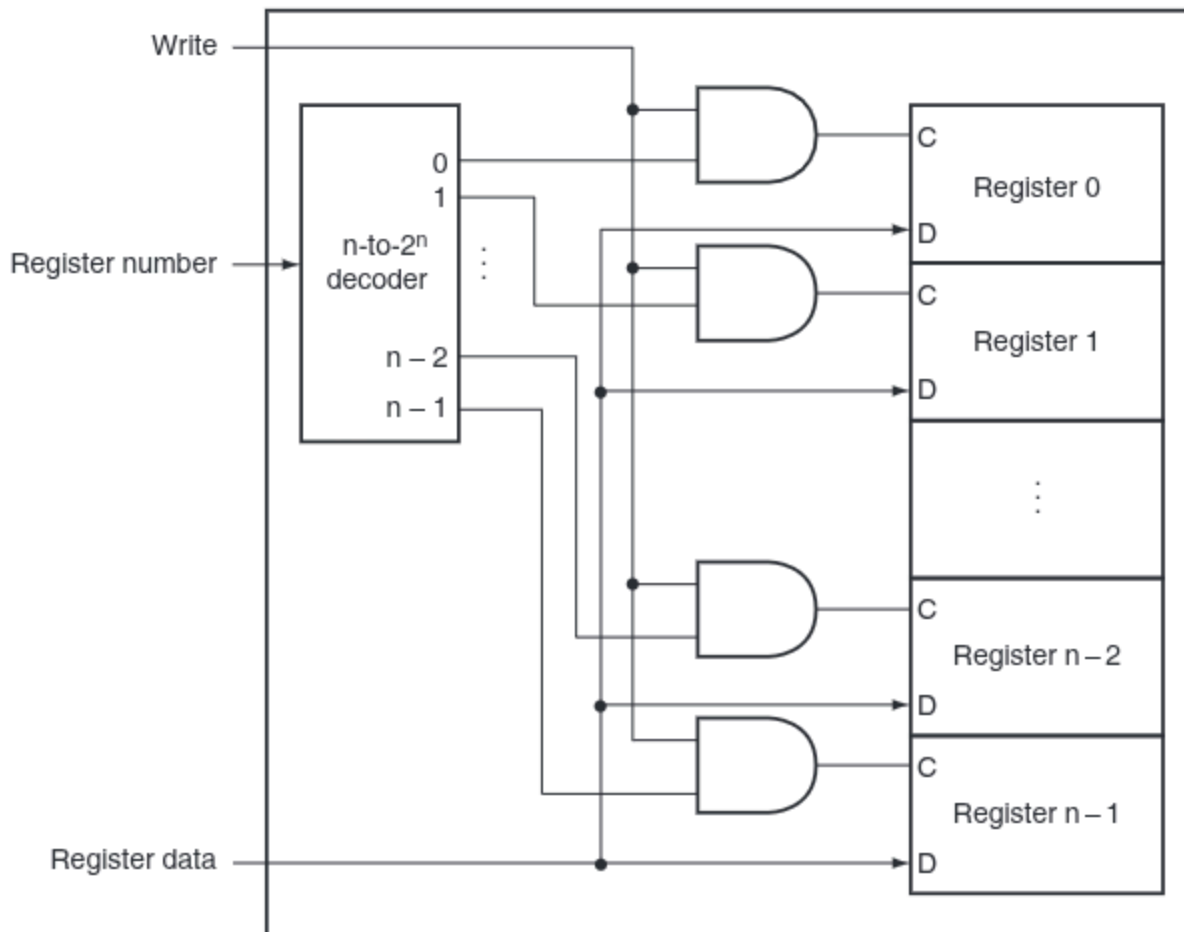


FIGURE A.8.9 The write port for a register file is implemented with a decoder that is used with the write signal to generate the C input to the registers. All three inputs (the register number, the data, and the write signal) will have setup and hold-time constraints that ensure that the correct data are written into the register file.

3. Forklare tilstandsmaskin maskin

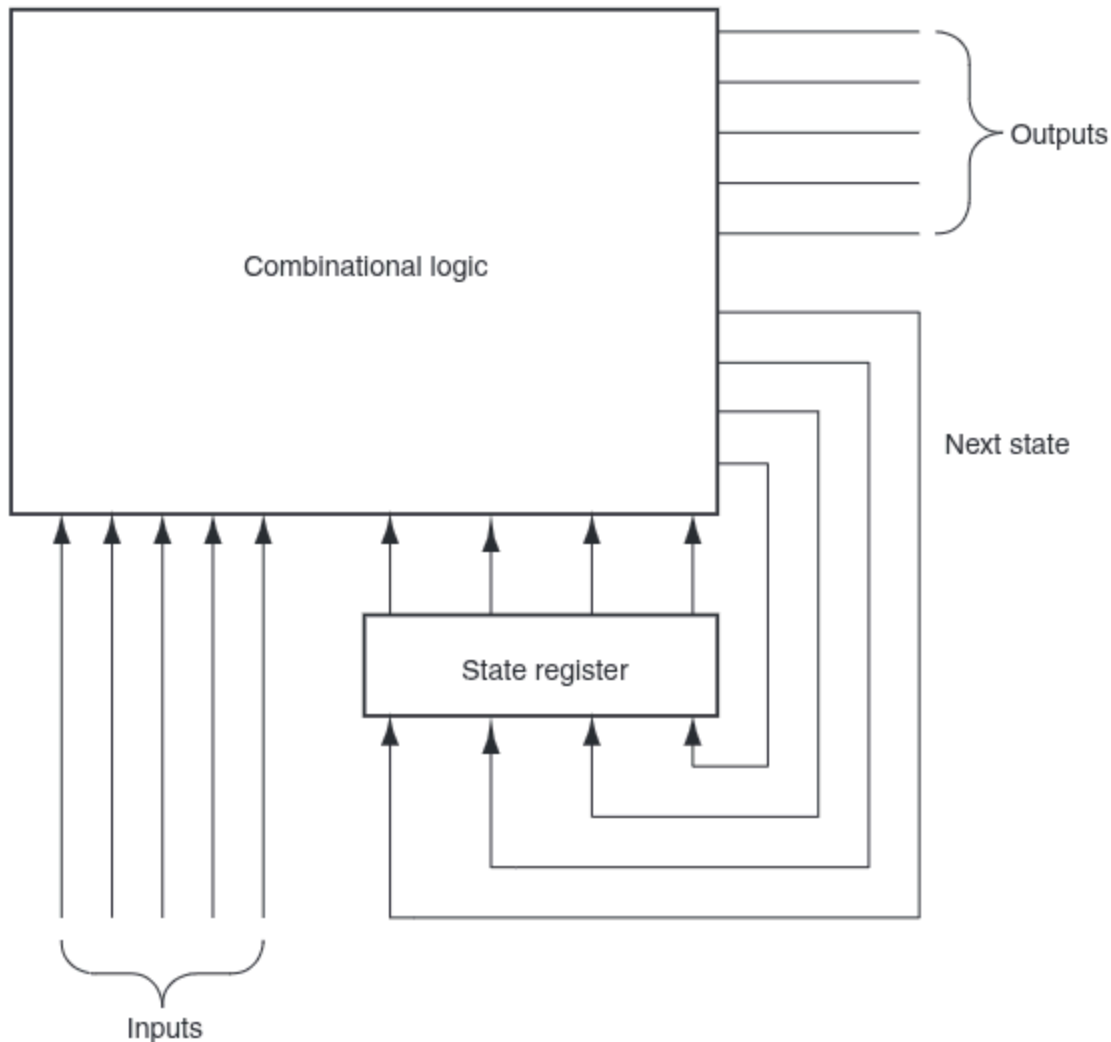


FIGURE A.10.3 A finite-state machine is implemented with a state register that holds the current state and a combinational logic block to compute the next state and output functions. The latter two functions are often split apart and implemented with two separate blocks of logic, which may require fewer gates.

4. Klokkesignal og "kritisk sti"

- Klokkesignal er et signal med fast frekvens som bytter mellom en satt høy og lav verdi, typisk 0-1V eller 0-3.3V f.eks.
 - Klokkesignalet brukes for å synkronisere delsystemer i et synkront system ved at de oppdaterer seg kun ved enten stigende eller fallende flanke på klokkesignalet.
 - Høyere spenning og frekvens vil føre til økt effektforbruk, så er ofte ønsket å ha så lav spenning som mulig.
- "Kritisk sti"
 - Beskriver den datastien som bruker lengst tid å propagere gjennom.

- Begrenser maksimale klokkefrekvensen, ettersom perioden kan ikke være kortere enn lengden på den kritiske stien.
- Tar tid å gå gjennom D-vipper og kombinatorisk logikk, summen av de i serien til kritiske stien vil bli den minste perioden.

T5 Samlebåndprosessorer

T5.1 Samlebåndprosessor med 5 steg

1. Enkeltsykel, flersykel og samlebåndprosessor

- Enkeltsykel
 - Lang kritisk sti.
 - Må ha flere adderere og minner for å få alt i en klokkesykel.
 - Tar mer areal uten bedre ytelse.
 - Vanskelig med operasjoner som krever flere klokkesykler som divisjon.
 - Alle instruksjoner tar like lang tid som er ikke god bruk av "make the common case fast".
- Flersykel
 - Kan gjøres svært liten og billig.
 - Bruker flere klokkesykler på en instruksjon, CPI øker.
 - Mindre å gjøre per klokkesykel gjør at vi kan øke klokkefrekvens.
 - Ikke særlig god ytelse.
- Samlebåndprosessor
 - Bruker instruksjonsnivåparallelitet (ILP) som gir økt ytelse.
 - Har lav CPI og sykeltid, og kan gjøres litent og stort.
 - Flere steg i samlebåndet fører til høyere oppstartskostnad, må balansere antall steg mot antall instrukser som skal kjøres.
 - Stegene i samlebåndet bør ta ca like lange tid så klokkefrekvensen ikke må reduseres.

2. Oppstartskostnad og balanse for ytelse

- Det tar $n - 1$ klokkesykler for å fylle et n stegs samlebånd, som vil gi en maksimal ytelseforbedring på nx .

$$n = \frac{t_{\text{enkeltsykel}}}{t_{\text{samlebånd}}}$$

- Realistisk ytelsesforbedring vil oftest være lavere ettersom det ikke vil være balanse mellom stegne i samlebåndet.

3. Fire overordnede strategier for å håndtere farer

- Unngåelse (unngå ressurskonflikter)
 - Forhindrer med gode arkitekturvalg
 - Lager mulighet for å lese av 2 registre og skrive inn et register i samme klokkesykel
- Videreending:
 - Forhindrer datafarer
 - Sender verdier videre fra enten ALU (EX/MEM), data minne (MEM/WB) eller som vanlig fra registeret.
- Stans:
 - Forhindrer les-bruk farer
- Prediksjon:
 - Forhindrer kontrol farer.
 - Antar at vi ikke bytter når sender inn en branch.
 - Legger til ekstra maskinvare for å redusere forsinkelsen når vi faktisk bytter branch og for å fjerne instruksjoner som ikke skulle kjøres men som er startet (flush).
 - Kan bruke en branch prediction buffer for å bestemme om vi skal anta å branche eller ikke basert på tidligere inputs.

4. Forklare mikroarkitektur til samlebåndsprosessor

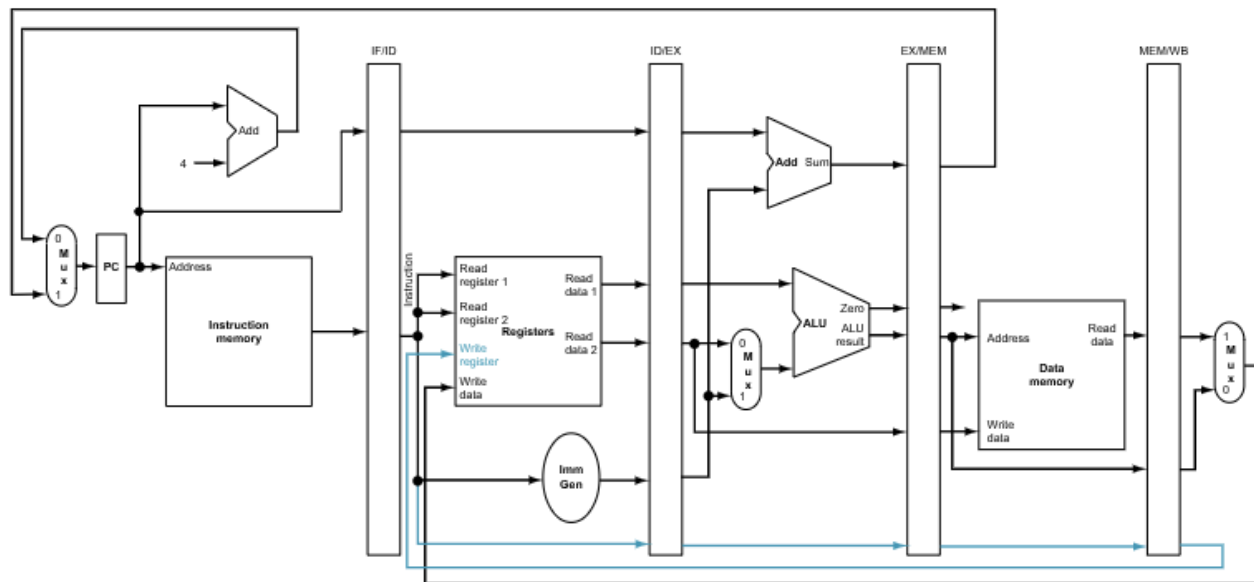


FIGURE 4.43 The corrected pipelined datapath to handle the load instruction properly. The write register number now comes from the MEM/WB pipeline register along with the data. The register number is passed from the ID pipe stage until it reaches the MEM/WB pipeline register, adding five more bits to the last three pipeline registers. This new path is shown in color.

5. Forklare kontrollenhet implementeres i 5 stegs samlebpnd

T5.2 Unntak og avbrudd

1. Forklar unntak, avbrudd og presise unntak

- Unntak:
 - Uforvenede endringer i kontrollflyten som kan være grunnet interne eller eksterne grunner.
- Avbrudd:
 - Brukes kun for unntak forutsaket av eksklusivt eksterne kilder som IO.
- Når et unntak dukker opp må maskinvaren/prosessoren:
 - Lagre adressen til instruksjonen som utføres i et bestemt register (SEPC)
 - Lagre grunnen til at unntaket i et bestemt register (SCAUSE)
 - Overføre kontroll til operativsystemet på en bestemt adresse
 - La operativsystemet behandle unntaket.
- Presise unntak:
 - Alle instruksjoner før instruksjonen som kastet unntaket må fullføres og alle andre instrukser etter må fjernes (flushes), slik at programmet kan fortsette kjøring etter unntaket.
 - Utnyttes blant annet for å implementere virtuelt minne.

2. Implementere presise unntak i 5 stegssamlebåndet

- Vi implementerer i kontrollenheten muligheten for å flushe ut instruksjonene.
- I tillegg må legges opp for at instruksjonen som utføres lagres i SEPC, og at instruksjonen som utløste unntaket lagres i SCAUSE.

3. Presise unntak i ut-av-rekkefølge

- For en ut-av-rekkefølge samleband så kan vi bruke en "Re-Order Buffer" (ROB) og "Register Renaming".
 - ROB vil holde styr på rekkefølgen til instruksene og sørge for at når et unntak blir kastet så vil kun instruksjonene før unntaket få bli kjørt.
 - RR vil holde styr på verdiene til de ulike registrene som instruksjonene spesifiserer og unngår WAW og WAR farer.

T5.3 Prosessorer med høyere ytelse

1. Parallellitet i tid og rom

- Prosessorer kan utnytte parallellitet i tid ved å kjøre flere instruksjoner samtidig som i samlebands arkitekturen. Da deler en opp datastien i ulike sekvenser som flere

instrukser kan propagere gjennom samtidig,

- Prosessorer kan utnytte parallellitet i rom ved å kjøre flere instrukser i samme sekvens (superskalaritet). Krever mye ekstra programvare å gjennomføre og fører til ekstra datavhengigheter og farer.

2. RAW, WAW & WAR farer

- Read after write:
 - Dersom vi leser av et register før det vi har skrevet til registere rekker å bli lagret.
 - Vil da lese av den gamle verdien av registeret, og ikke den nye skrevne verdien.
- Write after write:
 - Når utfører flere instruksjoner samtidig så kan det dukke opp navnavhengigheter som gjør at flere instrukser vil skrive til samme register. Dersom da ene skal skrive til før den andre, men den andre tar kortere tid kan rekkefølgen til instruksene bli feil.
- Write after read:
 - Når utfører flere instruksjoner samtidig så kan det hende at en instruks vil skrive til et register før en tidligere instruks rekker å lese register verdien.

3. Hvordan "Register renaming" fjerner WAW og WAR farer

- Register renaming vil si at vi legger til flere registre enn hva vi programmet vårt vet om, slik at vi kan omdøpe registre som har navnavhengigheter som fører til WAW og WAR farer.
- F.eks to instruksjoner som begge vil skrive til x3 blir døpt om til å skrive til ekstra registre p5 og p6.

4. Fordeler og ulemper med "Static multie-issue" prosessorer

- Kan kjøre flere instrukser helt samtidig som gir økt ytelse.
- Krever at enten kompilatoren tar en del ansvar for å redusere/unngå farer eller at hardwaren forsinker instrukser dersom dukker opp.
- Må kunne kjøre f.eks. to instrukser samtidig i en two issue pipeline, hvor ene må erstattes med en *nop* om ikke kan lages et instruksjonspar.
- Krever ekstra datalinjer, og f.eks. ekstra inn/utganger til registrene, immediate generator og ALU for å lage en static two issue pipeline.

5. Prinsippet bak ut av rekkefølge implementeres

- Instruksjoner går gjennom som vanlig til de kommer inn i rename registeret.
 - Der blir WAW og WAR farer fjernet ved å omdøpe registrene brukt slik at får kopier av alle dataene og unngår navnavhengigheter.
- Deretter blir instruksene sendt til Dispatch som holder instruksjonen til den sendes inn i riktig issue buffer, enten for å gjøre en ALU eller minne operasjon. Kan da gjøre

minne og ALU operasjoner samtidig ved å ha 2 paralelle tråder.

- Rekkefølgen på instruksene blir lagret i Reorder Bufferet som passer på at rekkefølgen instruksjonene blir sendt ut av WB er samme som de kom inn i.

6. Forklare spekulasjon

- Fra flotte ideen om prediksjon har vi spekulasjon som lar kompilatoren gjette på egenskapene til en instruksjonen, slik at den kan starte å kjøre instruksjoner avhengige av den spekulerende instruksjonen tidligere. Dette fører til høyere ytelse ved med instruksjonsnivåparallellisme.

T6: Minnesystemet

T6.1 Minnehierarki og hurtigbuffer

1. Lokalitet i tid og rom, og hvordan oppstår

- Lokalitet oppstår som en konsekvens av hvordan vi programmer. I minnehierarkiet vil det ofte være at om vi aksesserer et dataelement, så vil vi sikkert bruke det igjen snart (lokalitet i tid) og at vi vil bruke dataelementen i nærheten i minnet (lokalitet i rom).

2. Hierarki av minner gir illusjon av stort og raskt minne.

- Lokalitet og hierarki av minner brukes for å skape illusjonen om at vi har et veldig stort og raskt minne.
 - Når vi veldig ofte har dataelementene lett tilgjengelig i det lille raske minnet på grunn av lokalitet, så vil vi på snitt bruke veldig lite tid selv om det tar veldig lang tid å hente nye dataelementer fra lengre bak i minnet.

3. Konstruere direktetilordnet hurtigbuffer og utvide til å håndtere hurtigbufferblokker på mer enn ett ord

Direktetilordnet hurtigbuffer kan hver adresse bare ligge på et sted i cachen. Har samme adresse som i hovedminnet, men deler det opp i byte offset, index og tag.

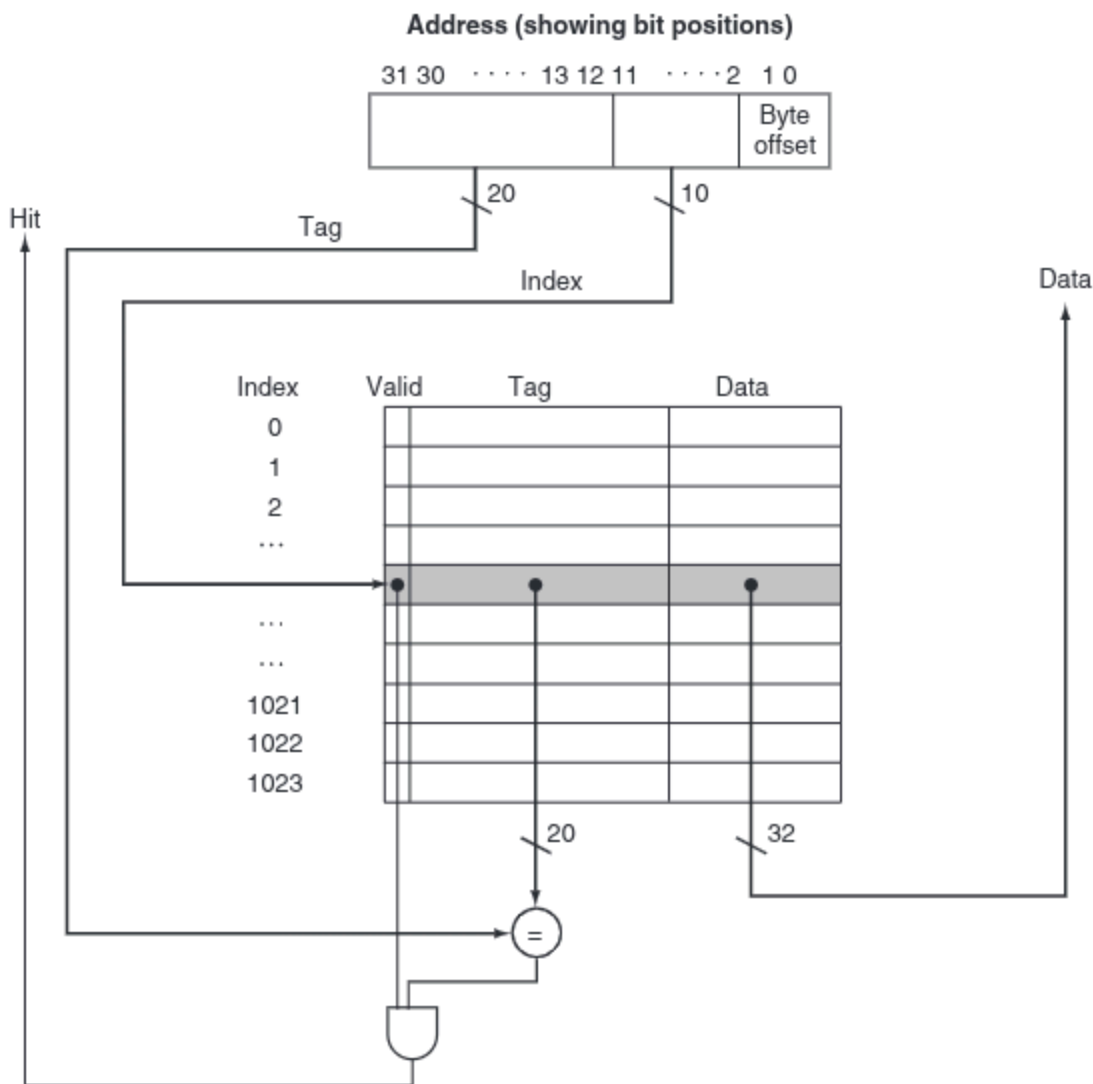


FIGURE 5.10 For this cache, the lower portion of the address is used to select a cache entry consisting of a data word and a tag. This cache holds 1024 words or 4 KiB. Unless noted otherwise, we assume 32-bit addresses in this chapter. The tag from the cache is compared against the upper portion of the address to determine whether the entry in the cache corresponds to the requested address. Because the cache has 2^{10} (or 1024) words and a block size of one word, 10 bits are used to index the cache, leaving $32 - 10 - 2 = 20$ bits to be compared against the tag. If the tag and upper 20 bits of the address are equal and the valid bit is on, then the request hits in the cache, and the word is supplied to the processor. Otherwise, a miss occurs.

Kan korte ned på index og legge til block offset for å legge til flere ord per hurtigbufferblokk. Da vil vi kunne hente ut flere ord for hver gang en aksesserer minne som gjør at får større sansynlighet for å treffe på dataelementer lokalt i rom.

4. Hurtigbufferne integrert i samlebåndsarkitektur

Kan ha et hurtigbuffer for instruksjonene i IF og for data i MEM.

- Dersom en treffer i et lite nok hurtigbuffer i samlebåndet så vil en kunne hente ut dataelementer direkte fra hurtigbufferet innen klokkesyklusen og fortsette som tidligere.

- Dersom en bommer må en stanse samlebandet til dataelementet blir hentet fra minnet før en kan fortsette instruksjonen.

5. Skrive til hurtigbuffer

Når det skrives til hurtigbufferet må det også oppdateres i minnet, brukes to hovedmetoder:

- Write-through: Hver gang skrives til hurtigbufferet så skrives det og gjennom til minnet.
- Write-back: Når et dataelement blir kastet ut av hurtigbufferet så oppdateres minnet. Må da vite om hurtigbufferblokk har blitt skrevet til med en dirty bit som indikerer at den har blitt endret.

Dersom vi skulle bomme på skriveoperasjonen, altså adressen vi vil skrive til ikke er lagret i hurtigbufferet har vi to til metoder:

- No write allocate: Skrive rett til minnet
- Write allocate: Henter dataelementet til hurtigbufferet og skriver til den i hurtigbufferet.

6. Beregne hvordan minneaksesstid påvirker ytelse

Se hvor mange instruksjoner som aksekkerer forskjellige nivåene av minnet og hvor lang tid det tar. F.eks:

- 90% treffer i 1. nivå minne (hurtigbuffer) og bruker 1 sykel.
- 8% treffer i 2. nivå minne og bruker 10 sykler.
- 2% treffer i 3. nivå minne og bruker 100 sykler.
 - Får da at:

$$1 \cdot 0.9 + (1 + 10) \cdot 0.08 + (1 + 10 + 100) \cdot 0.02 = 4.0$$

- Gjennomsnittlige minnelatens blir da 4 sykler.

$$CPI = \frac{\text{Sykler (beregning)}}{\text{Instruksjoner (total)}} + \frac{\text{Sykler (minne)}}{\text{Instrukser (total)}}$$

7. Beregne treffraten i flernivå hurtigbuffer via aksesstiden

^^^

8. Hvorfor sett- og fullassosiative hurtigbuffer øker treffraten

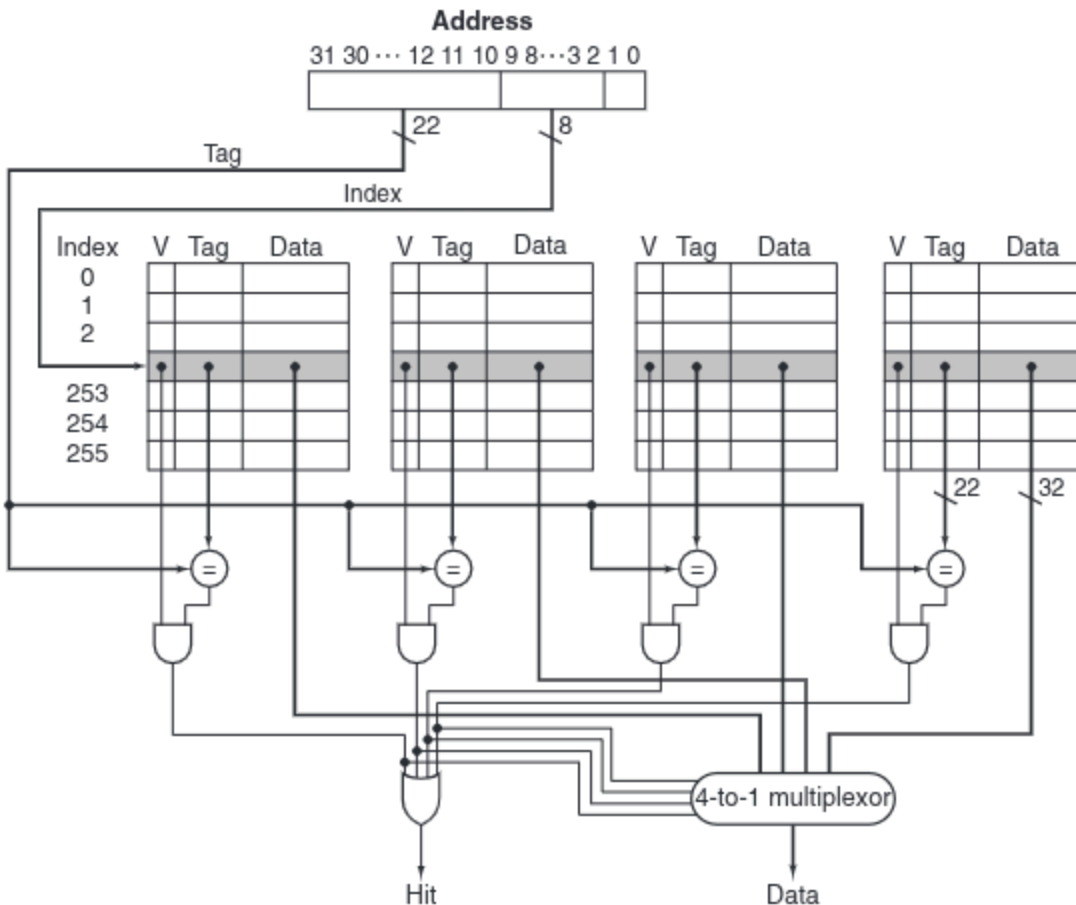


FIGURE 5.18 The Implementation of a four-way set-associative cache requires four comparators and a 4-to-1 multiplexor. The comparators determine which element of the selected set (if any) matches the tag. The output of the comparators is used to select the data from one of the four blocks of the indexed set, using a multiplexor with a decoded select signal. In some implementations, the Output enable signals on the data portions of the cache RAMs can be used to select the entry in the set that drives the output. The Output enable signal comes from the comparators, causing the element that matches to drive the data outputs. This organization eliminates the need for the multiplexor.

Med sett- og fullassosiative hurtigbufferer kan vi velge mellom flere plasser å plassere dataelementene i hurtigbufferet. Kan da i settassosiative velge mellom et av settene og plassere for index verdien i de, eller for en fullassosiative kan du plasse en blokk hvor som helst, men er ekstra dyrt for må sammenligne med alle tagsene i hurtigbufferet. For å bestemme hvilken en velger er det vanlig å bruke den minst nylig brukte (LRU) som passer på hvor lenge siden de forskjellige hurtigbufferblokkene har blitt aksessert.

9. Øke treffraten med programvare

- Ved bruk av blokking kan en øke treffraten.
 - Går ut på å øke lokaliteten ved å redusere antallet minneaksesser i programmet mellom hver aksess til samme data.
 - AKA skrive bedre kode som tar mer hensyn til minne og gjenbraker data mer effektivt.

T6.2 Minne-teknologier

1. Volatilt vs ikke-volatilt minne og statisk vs dynamisk minne

- Volatilt minne
 - Mister verdiene lagret når mister strøm
 - RAM: Random access memory
 - Tar like lang tid å aksessere alle elementene i minnet
 - SRAM og DRAM
 - SRAM: Static Random Access Memory. Bygges opp av raske registrerte med Tristate buffer for å kunne koble flere utganger sammen, og deles opp i matriseform for å få en enklere dekode.
 - DRAM: Dynamic Random Access Memory. Lagrer dataen i en kondensator som vil lade seg ut over tid og når leser av. Må derfor jevnlig skrive tilbake til minne før lader seg for mye ut. Billigere og kan lagre flere verdier på samme areal enn SRAM. Aksesseres ved å sende adressen i to skritt, til raden og kolonnen til minnet. Kobles ofte separat fra prosessoren via minnebus, f.eks. DDR5 (Double Data Rate) som sender på positiv og negativ flanke.
- Ikke-volatilt minne
 - Beholder verdiene selv om mister strøm.
 - Flash lagring og magnetisk disk
 - Flash er raskere å lese og skrive til for er helt elektronisk, men er begrenset hvor mange ganger en kan skrive til forskjellige delene av minnet. Forhindrer dette ved å skrive jevnt ut til hele minnet.
 - Magnetisk disk bruker lengre tid for må mekanisk bevege seg til riktig spor og snurre det rundt for å lese/skrive. Bitene lagres i magnetisk materiale og kan skrives til uten problemer. God for lagring av store mengder data som ikke må kunne hentes spesielt raskt som langtidslagring.

2. Kostnad av minne og bruk i hierarkiet

- Kostnad og plassering i hierarkiet går ettersom
 - 1. SRAM, raskest men dyrest å gjøre stort
 - 2. DRAM, ganske raskt og relativt stort
 - 3. Flash, relativt raskt og ganske stort
 -

4. Magnetisk disk, ganske tregt men svært stort og relativt billig.

3. Konstruere SRAM

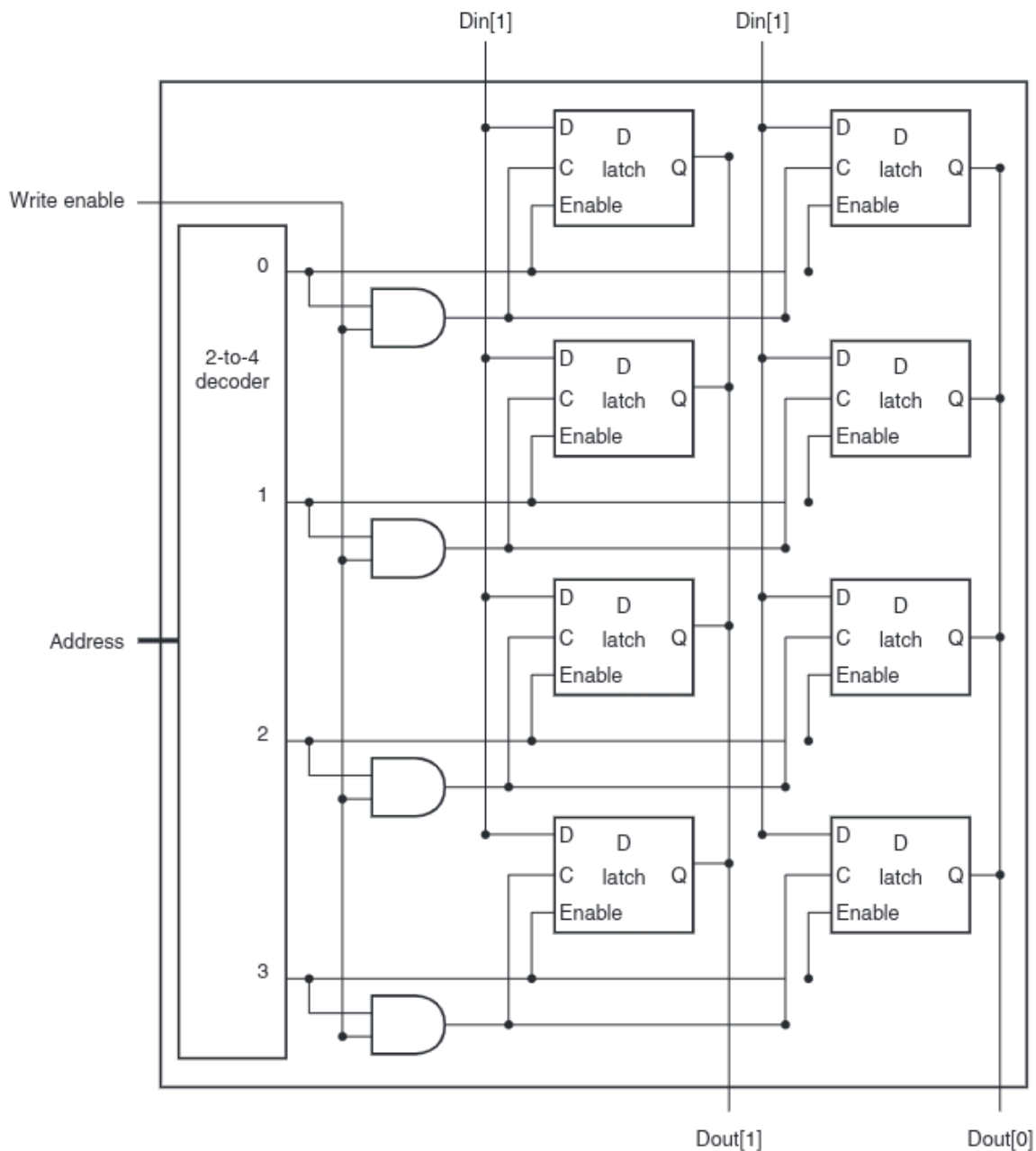


FIGURE A.9.3 The basic structure of a 4×2 SRAM consists of a decoder that selects which pair of cells to activate. The activated cells use a three-state output connected to the vertical bit lines that supply the requested data. The address that selects the cell is sent on one of a set of horizontal address lines, called word lines. For simplicity, the Output enable and Chip select signals have been omitted, but they could easily be added with a few AND gates.

Ut fra decoderen kommer om skal skru på kolonnene (Enable) for å lese av verdien. Dersom Write enable er på vil det også sendes høy til klokken på registeret (C) og inngangen (D) vil propagere til utgangen Q.

4. Konstruiere DRAM

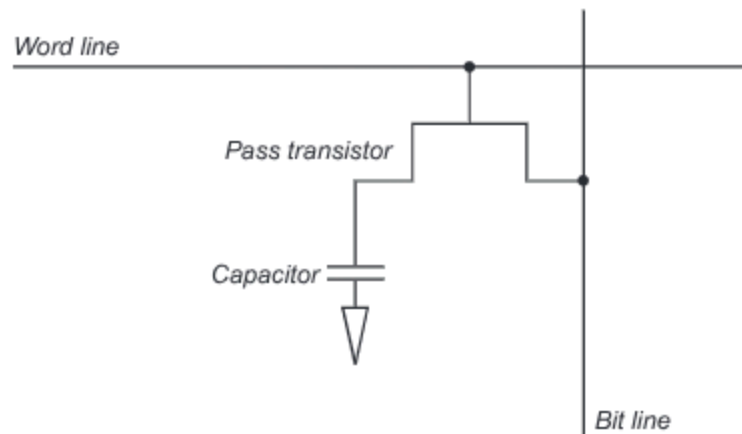


FIGURE A.9.5 A single-transistor DRAM cell contains a capacitor that stores the cell contents and a transistor used to access the cell.

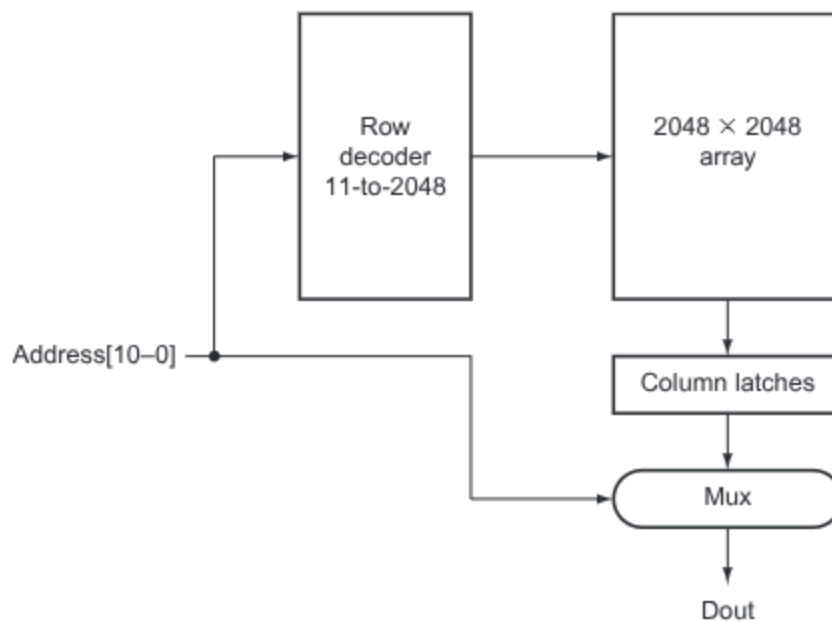


FIGURE A.9.6 A $4\text{M} \times 1$ DRAM is built with a 2048×2048 array. The row access uses 11 bits to select a row, which is then latched in 2048 1-bit latches. A multiplexor chooses the output bit from these 2048 latches. The RAS and CAS signals control whether the address lines are sent to the row decoder or column multiplexor.

T6.3 Virtuelt minne

- 1.
- 2.
- 3.
- 4.
- 5.

T7: Parallele datamaskiner

T7.1 Ytelse og Flynnns taksonomi

- 1.
- 2.
- 3.
- 4.
- 5.

T7.2 Multiprosessorer

- 1.
- 2.
- 3.
- 4.
- 5.

T7.3 Akseleratorer (Grafikkprosessorer og domene-spesifikke akseleratorer)

- 1.
- 2.
- 3.
- 4.
- 5.